

Final Project - Presentation

Performance Analysis of Parallel GMRES Solvers for 2D Fluid Flow on
High-Performance Clusters

Sarju Poudel



May 05, 2025

- Goal: Parallel 2D fluid simulation using GMRES Solver
- Solve incompressible Navier–Stokes equations
 - Non-linear viscous term (Explicit)
 - Linear Diffusion and Pressure Term (Implicit)
 - Pressure Poisson Equation ($Ax=b$)
 - GMRES: Generalized Minimal Residual
- Execution Methods
 - Serial Solver
 - Shared Memory Parallelization: OpenMP
 - Distributed Memory Parallelization: MPI
- Focus on scalability and performance

- Navier–Stokes Equation:

$$\rho \left(\frac{\partial u}{\partial t} + (u \cdot \nabla)u \right) = -\nabla p + \mu \nabla^2 u + f \quad (1)$$

$$\nabla \cdot u = 0 \quad (2)$$

where $u = (u_x, u_y)$ is the velocity field, p is pressure, ρ is density, μ is dynamic viscosity, and f is an external force.

- Linear terms: $-\nabla p + \mu \nabla^2 u + f$; Nonlinear term: $(u \cdot \nabla)u$
- Explicit treatment (known from previous time step):
 - Advection: $(u \cdot \nabla)u$
 - Diffusion: $\nu \nabla^2 u$
 - Forcing: f (here $G = \frac{dp}{L_x}$)
- Implicit treatment (pressure projection):

$$\nabla^2 p = \frac{\rho}{\Delta t} \nabla \cdot u^* \quad (3)$$

- All explicit terms are computed using current values u^n, v^n (no linear solve required)

- Advection term:

$$(u \cdot \nabla)v \approx u^n \frac{\partial v^n}{\partial x} + v^n \frac{\partial v^n}{\partial y} \quad (4)$$

- Diffusion term:

$$\nu \nabla^2 u^n \approx \nu \left(\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \right) \quad (5)$$

- Forcing Term:

$$f \approx G = \frac{\Delta p}{L_x} \quad (6)$$

- Intermediate Velocity Update (explicit Euler step):

$$u^* = u^n - \Delta t(\text{advection}) + \Delta t(\text{diffusion}) + \Delta t(\text{forcing}) \quad (7)$$

- Pressure equation is solved using GMRES (linear system solver)
- Pressure is obtained by enforcing incompressibility on the intermediate velocity u^* .

$$\nabla^2 p = \frac{\rho}{\Delta t} \nabla \cdot u^* \quad (8)$$

- Discretized form:

The divergence of u^* is computed from the grid values:

$$\nabla \cdot u^* \approx \frac{u_{i+1,j}^* - u_{i-1,j}^*}{2\Delta x} + \frac{v_{i,j+1}^* - v_{i,j-1}^*}{2\Delta y}$$

The Laplacian of p is computed from grid values:

$$\nabla^2 p_{i,j} \approx \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} \quad (9)$$

- Generalized Minimal Residual (GMRES) method for solving pressure Poisson equation
- Discrete linear system form:

$$Ap = b \quad (10)$$

- Where:
 - p is the vector of all pressure values in the domain
 - $b = \frac{\rho}{\Delta t} \nabla \cdot u^*$ is the right-hand side computed from the velocity field
 - $A \equiv -\nabla^2$ is the discrete Laplacian operator (finite-difference stencil based on dx, dy)
- GMRES solver for $Ap = b$:
 - Initial guess:

$$p^{(0)} \approx 0 \quad (\text{or previous timestep solution}) \quad (11)$$

- Initial residual:

$$r^{(0)} = b - Ap^{(0)} \quad (12)$$

- Residual norm:

$$r_0 = \|r^{(0)}\| \quad (13)$$

- Krylov subspace construction:

$$\mathcal{K}_k(A, r^{(0)}) = \text{span}\{r^{(0)}, Ar^{(0)}, A^2r^{(0)}, \dots, A^{k-1}r^{(0)}\} \quad (14)$$

- At each iteration, GMRES:

- Applies matrix-free operator: $Av \equiv -\nabla^2 v$
- Builds an orthonormal basis of the Krylov space
- Minimizes the residual $\|b - Ap\|$ over this space

- Update step:

$$p^{(k)} \rightarrow p^{(k+1)} \quad \text{chosen to minimize } \|r^{(k)}\| \quad (15)$$

- Convergence criterion:

$$\frac{\|r^{(k)}\|}{\|r^{(0)}\|} < \text{tol} \quad (16)$$

- The computed pressure gradient is then used to project the velocity:

$$u^{n+1} = u^* - \frac{\Delta t}{\rho} \nabla p \quad (17)$$

$$u_x^{n+1} = u_x^* - \frac{\Delta t}{\rho} \frac{\partial p}{\partial x} \quad (18)$$

$$u_y^{n+1} = u_y^* - \frac{\Delta t}{\rho} \frac{\partial p}{\partial y} \quad (19)$$

- Serial Programming (Single Processor)
 - Solves full CFD domain on one core
 - Large sparse linear systems (pressure Poisson solve)
 - Limitations:
 - Computational time increases rapidly with grid size
 - Memory bottleneck for large domains
 - Not scalable for high-resolution simulations
- Shared Memory Parallelization: OpenMP
 - Parallel execution on multiple CPU cores
 - Shared memory architecture (single node)
 - Limited scalability due to memory bandwidth
- Distributed Memory Parallelization: MPI
 - Domain decomposition across multiple processes
 - Each process handles a subdomain of the grid
 - Requires communication (halo exchange) between neighbors
 - Enables large-scale CFD and pressure solve (GMRES)

- Distributed memory systems consist of multiple independent compute nodes
 - Each node has its own CPU and private memory (RAM)
 - No shared memory between nodes
- The global problem is divided into smaller subproblems. Example: Domain decomposition in CFD
- Each MPI process handles one subproblem (subdomain)
- Each process:
 - Stores only local grid data
 - Computes physics locally (advection, diffusion, pressure solve)
- Physical quantities are coupled across boundaries of neighboring subdomains
- MPI communication:
 - Processes send and receive boundary (halo/ghost cell) data
 - Enables consistent global solution across all subdomains

- In Computational Fluid Dynamics (CFD), the physical domain is continuous
- Governing equations (Navier–Stokes) are defined in continuous space:

$$\frac{\partial u}{\partial t}, \quad \nabla u, \quad \nabla^2 p \quad (20)$$

- To solve numerically, the domain is discretized into a finite grid
- Discretization converts PDEs into algebraic equations:
 - Derivatives $\rightarrow$ finite differences
 - Domain $\rightarrow$ collection of grid points
- Each grid point represents a control volume where variables are stored:
 - Velocity (u, v)
 - Pressure p

Each cell stores u, v, p

•	•	•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•	•	•

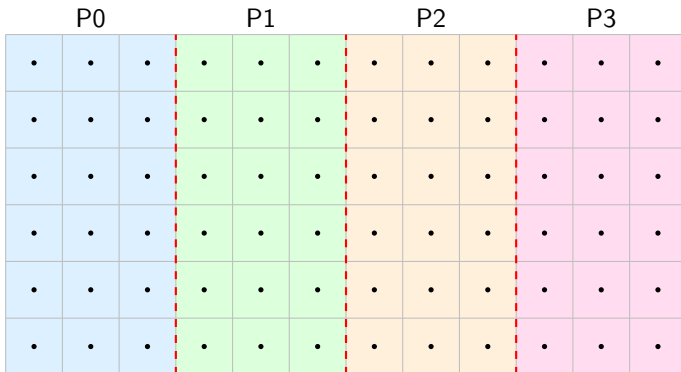
Discretized CFD domain (Finite Difference / Finite Volume)

- Goal: Split a large CFD grid across multiple MPI processes
- Each process computes a subdomain and exchanges boundary data (ghost cells)
- Two common strategies:
 - 1D Decomposition: Domain is split along one direction only; horizontal or vertical
 - Only 2 boundaries per process (left and right) or (top and bottom)
 - Fewer MPI communications: 2 Neighbors only
 - Poor scalability and Load Imbalance for non-uniform domains
 - 2D Decomposition: Domain is split in both x and y directions
 - 4 boundaries per process (left, right, top, bottom)
 - Better load balancing
 - Much better scalability for large CFD problems
 - Matches natural 2D structure of PDEs
 - High MPI Overhead; Requires communication with up to 4 neighbors

1D Domain Decomposition (MPI)



1D Domain Decomposition (4 MPI Processes)



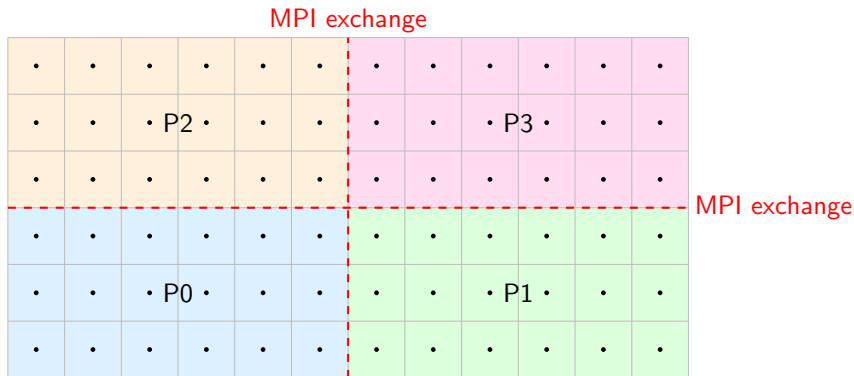
MPI exchange

Each process exchanges ghost cells only at interfaces

2D Domain Decomposition (MPI)



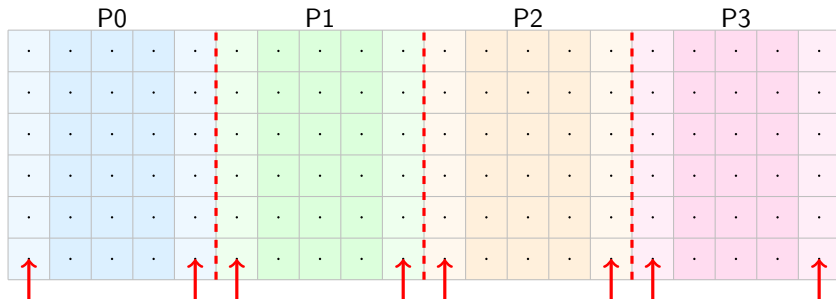
2D Domain Decomposition (4 MPI Processes)



Each process communicates with up to 4 neighbors via ghost cells

- In CFD, each grid point depends on its neighboring points (stencil operations)
- When using MPI, each process owns a subdomain and needs neighbor data at the boundaries.
- Extra layers of cells added around each subdomain Ghost Cells (Halo Regions)
 - Store copied values from neighboring MPI processes
 - Allow local computations as if full domain still exists
- Ghost Cells Updates:
 - MPI_Send / MPI_Recv (blocking communication)
 - MPI_Isend / MPI_Irecv (non-blocking communication)
- Key idea:
 - Ghost cells make each process "think" it has a full neighborhood
 - Enables correct stencil-based computations at subdomain boundaries

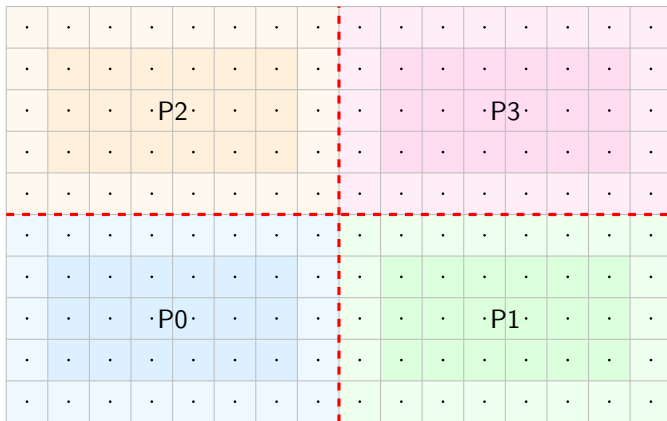
1D MPI Domain Decomposition with Ghost Cells



Red arrows indicate ghost cell regions (MPI halo exchange)

Light = ghost cells, Dark = interior computation cells

2D MPI Domain Decomposition with Ghost Cells



Ghost cells
exchanged via MPI

Light = ghost (halo), Dark = interior computation cells

1. Predictor Step (Explicit):

- Apply predictor boundary conditions
- Exchange ghost cells (velocity fields)
- Compute intermediate velocity:

$$u^*, v^* \text{ (advection + diffusion + forcing)}$$

2. Prepare Pressure Solve:

- Update pressure guess
- Exchange ghost cells for u^*, v^*
- Build RHS:

$$b = \frac{\rho}{\Delta t} \nabla \cdot u^*$$

3. Implicit Pressure Solve (GMRES):

- Solve linear system:

$$Ap = b, \quad A \equiv -\nabla^2$$

- Enforces incompressibility:

$$\nabla \cdot u^{n+1} = 0$$

4. Velocity Correction (Projection):

- Exchange pressure field (ghost cells)
- Update velocity:

$$u^{n+1} = u^* - \frac{\Delta t}{\rho} \nabla p$$

5. Finalization:

- Apply corrector boundary conditions
- Final halo exchange (u, v)
- Output diagnostics / VTK

- **1. Physics Layer (Boundary Conditions)**

- Inlet velocity specification (Dirichlet BC)
- No-slip wall conditions ($u = v = 0$)
- Outlet extrapolation (Neumann-type BC)

- **2. MPI Layer (Communication)**

- Ghost cells exchanged between neighboring subdomains
- Ensures stencil consistency across MPI ranks
- No physical boundary condition is applied here

- **3. Solver Layer (Numerics)**

- GMRES solves the pressure Poisson equation: $Ap = b$
- Pressure gradient used to correct velocity field
- Enforces incompressibility: $\nabla \cdot \mathbf{u}^{n+1} = 0$

- **Current output approach (from code):**
 - Field data is **distributed across MPI ranks**
 - Each rank stores local subdomain fields (u, v, p)
- **Gather-to-root strategy (implemented)**
 - Local fields are gathered on rank 0:
 - `gather_field_to_root(u, u_global, mesh)`
 - `gather_field_to_root(v, v_global, mesh)`
 - `gather_field_to_root(p, p_global, mesh)`
 - Rank 0 assembles full global field
 - Output written as VTK file
- **File organization (per run)**
 - Output directory:
 - `vtk_mpi/t<NUM_PROCS>/`
 - Time-step based files:
 - `output_mpi_0000.vtk, output_mpi_0001.vtk, ...`
 - Run log stored as:
 - `run.log` (rank 0 only)

- **MPI Initialization**

- MPI_Init
- Create MPI communicator and ranks

- **Simulation Setup**

- Initialize mesh and parameters (`initialize_mpi`)
- Allocate local and global arrays (rank 0)
- Create output directories and logging files

- **Domain Decomposition**

- Split global grid into local subdomains
- Assign each MPI rank a spatial region
- Setup ghost cell structure

- **Time-Stepping Loop**

- Boundary conditions (predictor/corrector)
- Ghost cell exchange
- Compute intermediate velocity
- Pressure solve (GMRES)
- Velocity correction
- Output (VTK every n steps)

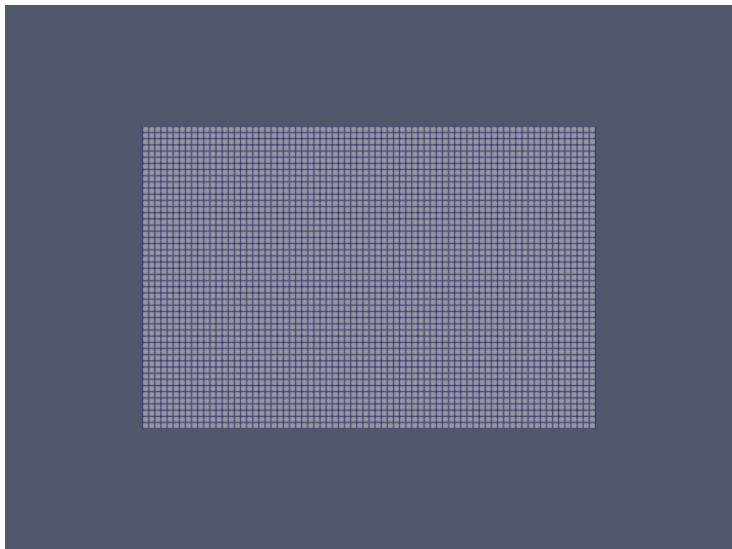
- **Finalization**

- Gather results on root process
- Write logs and final statistics
- Free memory and `MPI_Finalize`

- **Limitations of current approach**

- Communication bottleneck at root process
- Not scalable for very large simulations
- No parallel file writing (MPI-IO / HDF5 not used yet)

- **Missing or incorrect ghost cell updates**
 - Stale halo data leads to incorrect gradients
 - Breaks stencil consistency in ∇u , ∇p , and Laplacian
- **MPI communication errors (deadlocks / blocking issues)**
 - Mismatched MPI_Send / MPI_Recv or Missing synchronization in non-blocking communication
 - Rank dependency issues during halo exchange
- **Incorrect indexing in flattened 2D arrays**
 - Wrong mapping: $(i, j) \rightarrow i \cdot n_y + j$
 - Off-by-one errors near ghost cell boundaries
- **Inconsistent boundary treatment across ranks**
- **Load imbalance**
 - Unequal domain decomposition across MPI ranks
 - Some ranks idle during computation or GMRES
- **Solver-related issues**
 - GMRES breakdown due to near-zero denominator
 - Poor convergence from inconsistent operator A



Computational Domain: Discretized

Grid Parameters

- $n_x = 75, n_y = 50$
- $L_x = 1.5, L_y = 1.0$

Solver Parameters

- Max iter: 2000
- Tol: 10^{-4}

Physical Parameters

- $\nu = 0.05$
- $\rho = 1.0$

Boundary Conditions

- $U_{in} = 0.01$
- No-slip (Walls): $u = 0, v = 0$
- $p = 0$

Time

- $\Delta t = 10^{-5}$
- $N_T = 7500$

Output

- VTK: Off
- Every 25 steps

Execution Methods

- Serial
- Open MP (fine) [n=2, 4, 6]
- MPI [n= 2, 4 , 6]

Grid Parameters

- $n_x = 1536, n_y = 1024$
- $L_x = 1.5, L_y = 1.0$

Solver Parameters

- Max iter: 2000
- Tol: 10^{-4}

Physical Parameters

- $\nu = 0.05$
- $\rho = 1.0$

Boundary Conditions

- $U_{in} = 0.01$
- No-slip (Walls): $u = 0, v = 0$
- $p = 0$

Time

- $\Delta t = 10^{-5}$
- $N_T = 7500$

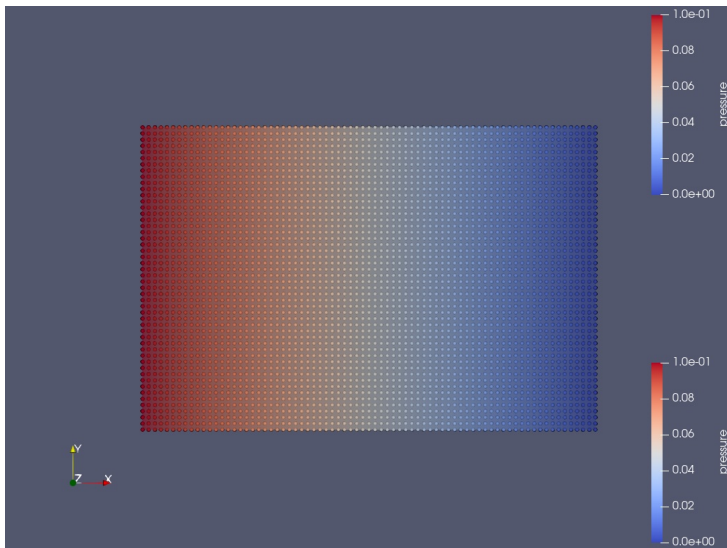
Output

- VTK: On
- Every 25 steps

Execution Methods

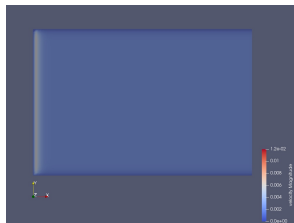
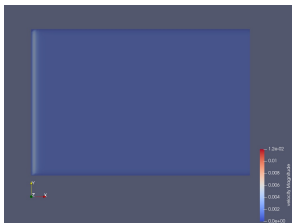
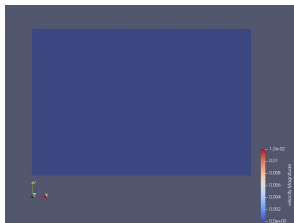
- Open MP (fine) [n=8, 16, 32, 64]
- MPI [n=8, 16, 32, 64]

Results: Pressure Field

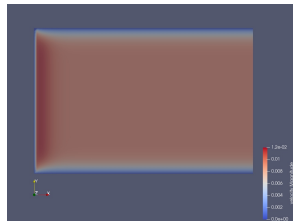
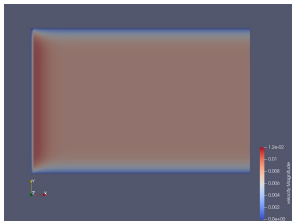
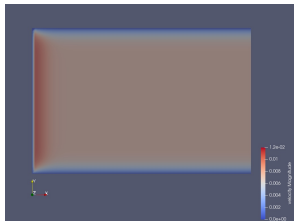
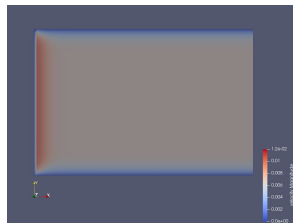
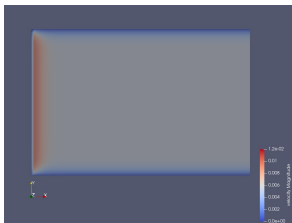
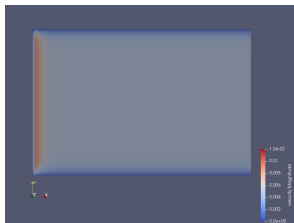


Pressure distribution across the computational domain

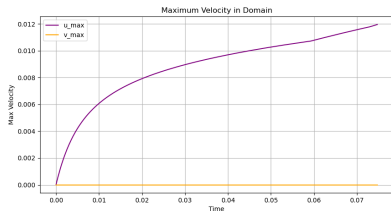
Results: Velocity



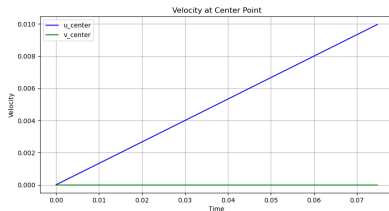
Results: Velocity



Results: Velocity



Maximum Velocity Evolution in Domain

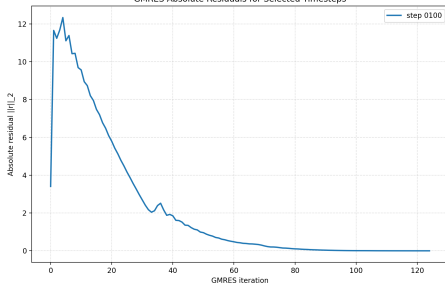


Velocity Evolution at Center Point

Results: GMRES Residual Convergence

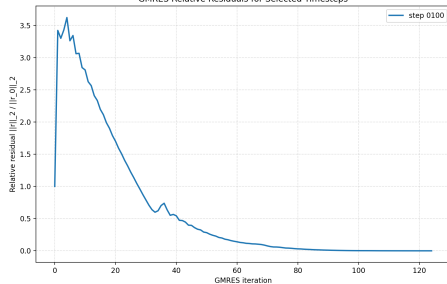


GMRES Absolute Residuals for Selected Timesteps



Absolute Residuals Convergence with
GMRES iteration

GMRES Relative Residuals for Selected Timesteps



Relative Residuals Convergence with
GMRES iteration

Results: Execution Time (Local Machine)



Case	Total (s)	Operator (s)	GMRES (s)	Op Calls	GMRES Calls
Serial	17.379	7.148	16.780	937500	7500
OpenMP (1)	22.412	10.158	21.687	937500	7500
OpenMP (2)	18.398	8.878	17.904	937500	7500
OpenMP (4)	15.036	7.419	14.667	937500	7500
OpenMP (6)	14.801	7.393	14.457	937500	7500
MPI (1)	17.388	7.046	16.907	937626	7500
MPI (2)	12.112	4.338	11.811	937626	7500
MPI (4)	9.209	2.863	9.012	937626	7500
MPI (6)	12.400	2.804	12.127	937626	7500

● Observation:

- Serial execution is faster than OpenMP for low thread counts (1–2 threads), due to parallel overhead.
- OpenMP begins to outperform serial execution at moderate thread counts (around 4 threads).
- Both OpenMP and MPI show performance improvement as the number of threads/processes increases, but only up to an optimal point.
- Beyond this point, performance degrades due to overheads such as thread management, communication, and memory contention.
- Conclusion: For small problem sizes, parallelization can introduce more overhead than benefit, leading to increased computational cost.

Results: Execution Time (Nova, Small Domain)



Case: 1 Node	Total (s)	Operator (s)	GMRES (s)	Op Calls	GMRES Calls
Serial	19.127	8.159	18.502	937500	7500
OpenMP (1)	20.912	9.313	20.222	937500	7500
OpenMP (2)	15.924	7.659	15.457	937500	7500
OpenMP (4)	13.937	7.003	13.575	937500	7500
OpenMP (6)	14.218	7.309	13.881	937500	7500
OpenMP (8)	15.543	8.362	15.202	937500	7500
OpenMP (16)	22.401	12.427	21.980	937500	7500
OpenMP (32)	37.497	20.469	36.876	937500	7500
OpenMP (64)	49.384	24.892	48.588	937500	7500
MPI (1)	18.596	8.186	18.039	937626	7500
MPI (2)	12.724	5.015	12.384	937626	7500
MPI (4)	9.504	3.336	9.286	937626	7500
MPI (6)	8.899	2.711	8.717	937626	7500
MPI (8)	8.122	2.402	7.963	937626	7500
MPI (16)	7.819	1.928	7.685	937626	7500
MPI (32)	10.925	2.016	10.752	937626	7500

- Observation:

- Execution time decreases with more processors up to an optimal point, then increases due to overhead.
- Conclusion: For small domains, parallelization can increase overall cost.

Results: Execution Time (Nova, Small Domain)



Case: 2 Nodes	Total (s)	Operator (s)	GMRES (s)	Op Calls	GMRES Calls
MPI (1)	18.588	8.187	18.034	937626	7500
MPI (2)	81.515	19.022	80.194	937626	7500
MPI (4)	82.520	16.541	81.392	937626	7500
MPI (6)	78.089	13.714	77.150	937626	7500
MPI (8)	79.826	3.187	78.984	937626	7500
MPI (16)	94.409	2.398	93.603	937626	7500
MPI (32)	88.098	2.225	87.313	937626	7500
MPI (64)	106.922	3.304	105.960	937626	7500

- Observation:

- Execution time increases when moving from 1 to multiple cores due to added communication overhead.
- Using multiple nodes introduces inter-node communication (e.g., over Ethernet), which slows MPI performance.
- Conclusion: If sufficient cores are available on a single node, using multiple nodes can degrade performance.

Results: Execution Time (Nova Cluster: Large Domain)



Case	Total (s)	Operator (s)	GMRES (s)	Op Calls	GMRES Calls
Serial	1374.062	907.681	1372.914	54027	27
OpenMP (1)	1378.914	913.710	1377.394	54027	27
OpenMP (2)	740.541	489.124	739.696	54027	27
OpenMP (4)	443.820	276.921	443.086	54027	27
OpenMP (8)	222.771	138.435	222.332	54027	27
OpenMP (16)	113.318	70.121	113.042	54027	27
OpenMP (32)	59.168	36.555	58.893	54027	27
OpenMP (64)	35.165	20.873	34.536	54027	27
MPI (8)	2696.677	1199.057	2660.404	1882500	7500
MPI (16)	1747.367	517.656	1728.287	1882500	7500
MPI (32)	984.383	284.712	973.672	1882500	7500
MPI (64)	877.651	277.549	867.017	1882500	7500

● Observation:

- OpenMP is working better with use of more no. of threads.
- MPI communication head is significant and code optimization is needed to reduced the time of run for larger domain.
- Conclusion: Parallelization must be down with careful planning of how and when to exchange the data and how to reduce the communication head.

- Developed a CFD solver for incompressible 2D Navier–Stokes equations using GMRES
- Implemented and compared three execution strategies:
 - Serial (baseline single-core implementation)
 - OpenMP (shared-memory parallelization)
 - MPI (distributed-memory domain decomposition)
- Key performance insight:
 - Serial: simple but not scalable for large grids
 - OpenMP: efficient on shared memory, limited by bandwidth and cores
 - MPI: best scalability for large problems, but communication overhead is critical
- Domain decomposition + ghost cell exchange are essential for MPI correctness and scalability
- GMRES used for pressure Poisson system across all implementations
- Overall: MPI-based framework is most suitable for large-scale CFD simulations on HPC clusters

Thank you!

Any questions?

Performance Analysis of Parallel GMRES Solvers for 2D Fluid Flow on
High-Performance Clusters

Sarju Poudel